

MEMORY FOR AI AGENTS

What is **memory**?

“The ability to retain information or a representation of past experience, based on the mental processes of **learning or encoding**, **retention** across some interval of time, and **retrieval** or reactivation of the memory.”

APA Dictionary of Psychology, “memory”

Encode

turn an interaction into something storable



Store

keep it across sessions, at scale



Retrieve

bring back the right part, at the right time

TWO KINDS OF MEMORY

Some memories are **told**. Others are **inferred**.

Explicit memory

STATED → STORED

The user says a fact outright. Memory captures it as written.

I currently work at HydraDB and build AI systems there.

↓ extract facts

MEMORY STORE

Works at HydraDB. **STATED**

Builds AI systems. **STATED**

Implicit memory

OBSERVED → INFERRED

No single message says it. The pattern across sessions does.

MON · 6:12 PM

What are some good places around for cakes?

SAT · 9:40 AM

When does Ghirardelli open on the weekend?

⋮ infer across sessions

MEMORY STORE

Has a sweet tooth. **INFERRED**

Based on: cake places (Mon), Ghirardelli hours (Sat)

STORING AND RETRIEVING MEMORY

Three ways to give an agent memory

01

A memory file

Append facts to MEMORY.md, load the whole file into every prompt.

Best for a handful of preferences

Breaks when the file outgrows the context window

02

Vector store + chunks

Chunk everything, embed it, fetch the chunks closest to the question.

Best for pinpoint factual lookups

Breaks on time, contradictions and big-picture questions

03

A graph

Extract entities and relations, connect them over time, traverse to answer.

Best for evolving, connected memory

Holds accuracy as memory grows

01 · THE MEMORY FILE

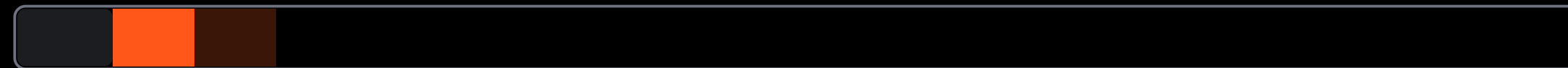
MEMORY.md works, until it doesn't

MEMORY.md

WHAT THE MODEL SEES ON EVERY TURN



Day 1
40 lines



fits easily



Month 3
4,000 lines



slow, costly,
facts get lost



Year 1
50,000+ lines



exceeds the window

system prompt
 the whole memory file
 the user's question
 context window

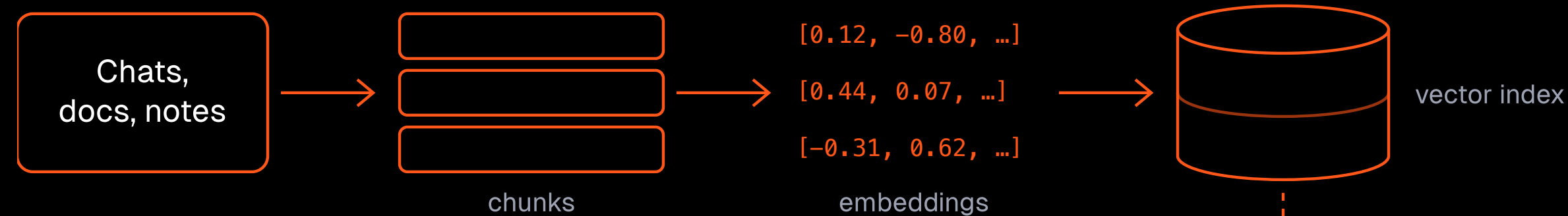
Good: zero infrastructure, perfectly transparent, great for a handful of preferences.

Breaks at scale: every turn pays for every memory; the context bloats, then the window overflows.

02 · VECTOR STORE + CHUNKING

Chunk it, embed it, **hope** it comes back

INGEST



QUERY



Great at

Pinpoint questions whose answer sits verbatim in one chunk: “Which port is the memory store on?”

Struggles with

Anything that needs time, consistency or the big picture (next two slides).

02 · WHY VECTORS GET NOISY AT SCALE

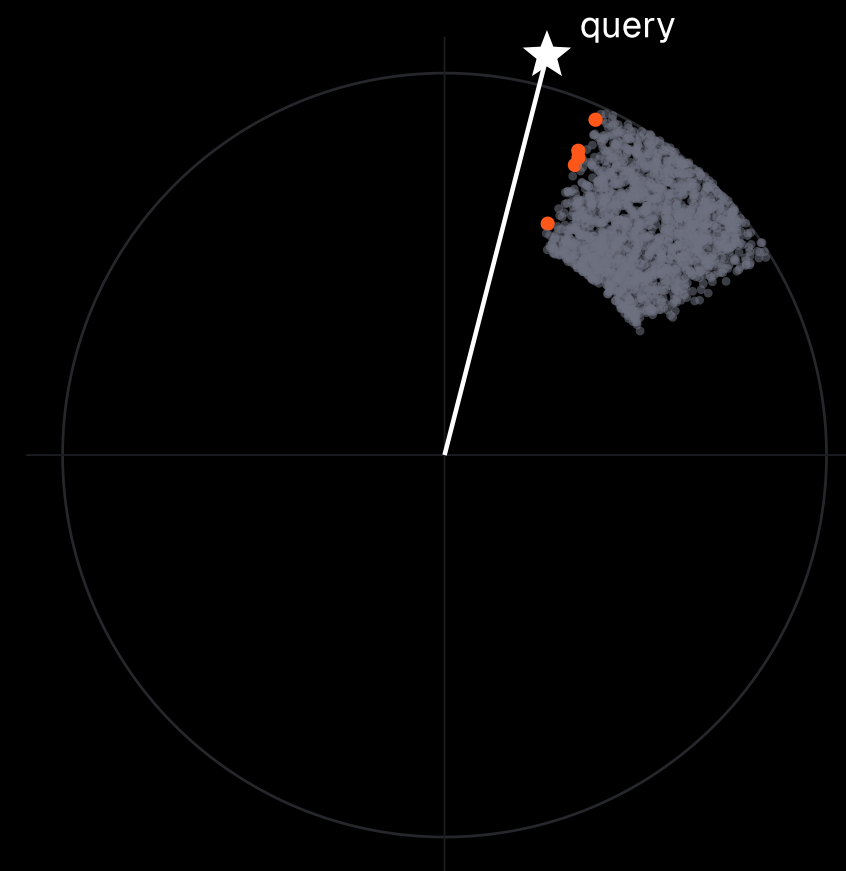
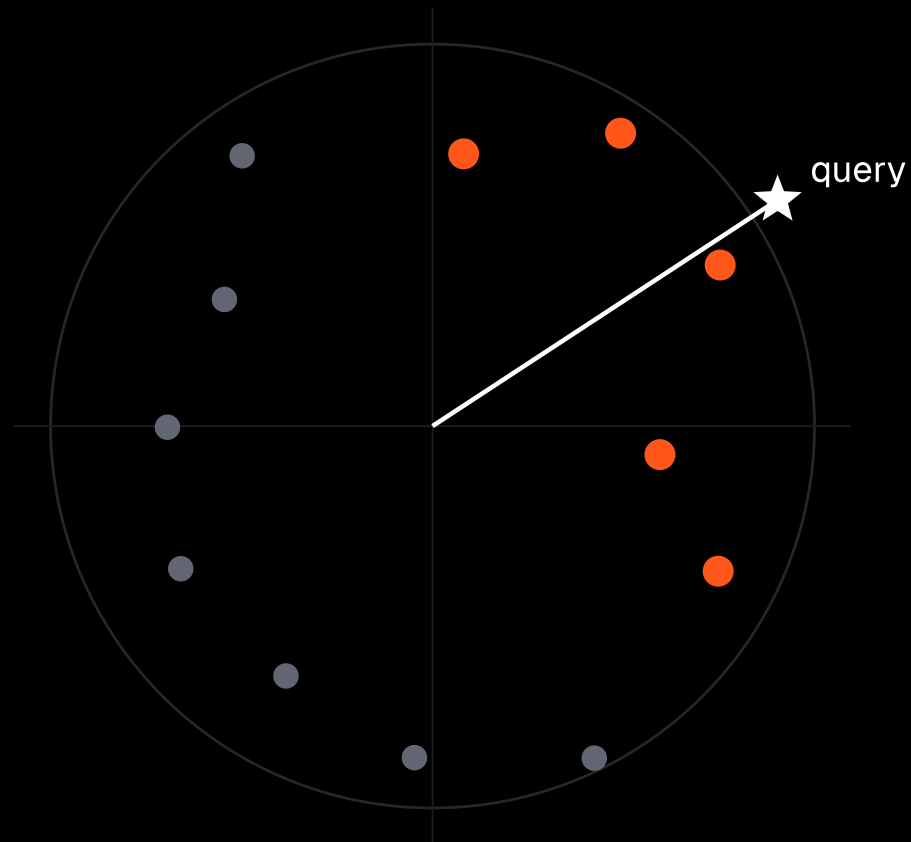
More memories, less separation

A few hundred chunks

CLEAR WINNER

Hundreds of thousands

PHOTO FINISH



Real embeddings are anisotropic: they bunch into a narrow cone. As the store grows, the top-k neighbours of a query end up a hair apart in cosine, so a slightly different phrasing returns a different set. (Illustrative 2-D projection; scores are the true cosines of the plotted points.)

02 · WHERE VECTOR MEMORY BREAKS

Similarity measures **relevance**, not answerability

Time

“Deadline is June 25” and
“deadline moved to July 2”
look equally relevant. Nothing
says which is current.

WHAT A GRAPH ADDS

Edges carry timestamps, so the
newest fact can win.

Staleness

Old facts are never retired, so
they keep winning retrieval
long after they stopped being
true.

WHAT A GRAPH ADDS

A superseded relation is marked
as such, not just outranked.

Contradictions

“I’ve shipped it” vs “I’ve never
built it”: both come back, with
no notion that they conflict.

WHAT A GRAPH ADDS

Both claims attach to the same
entity, so they surface side by
side.

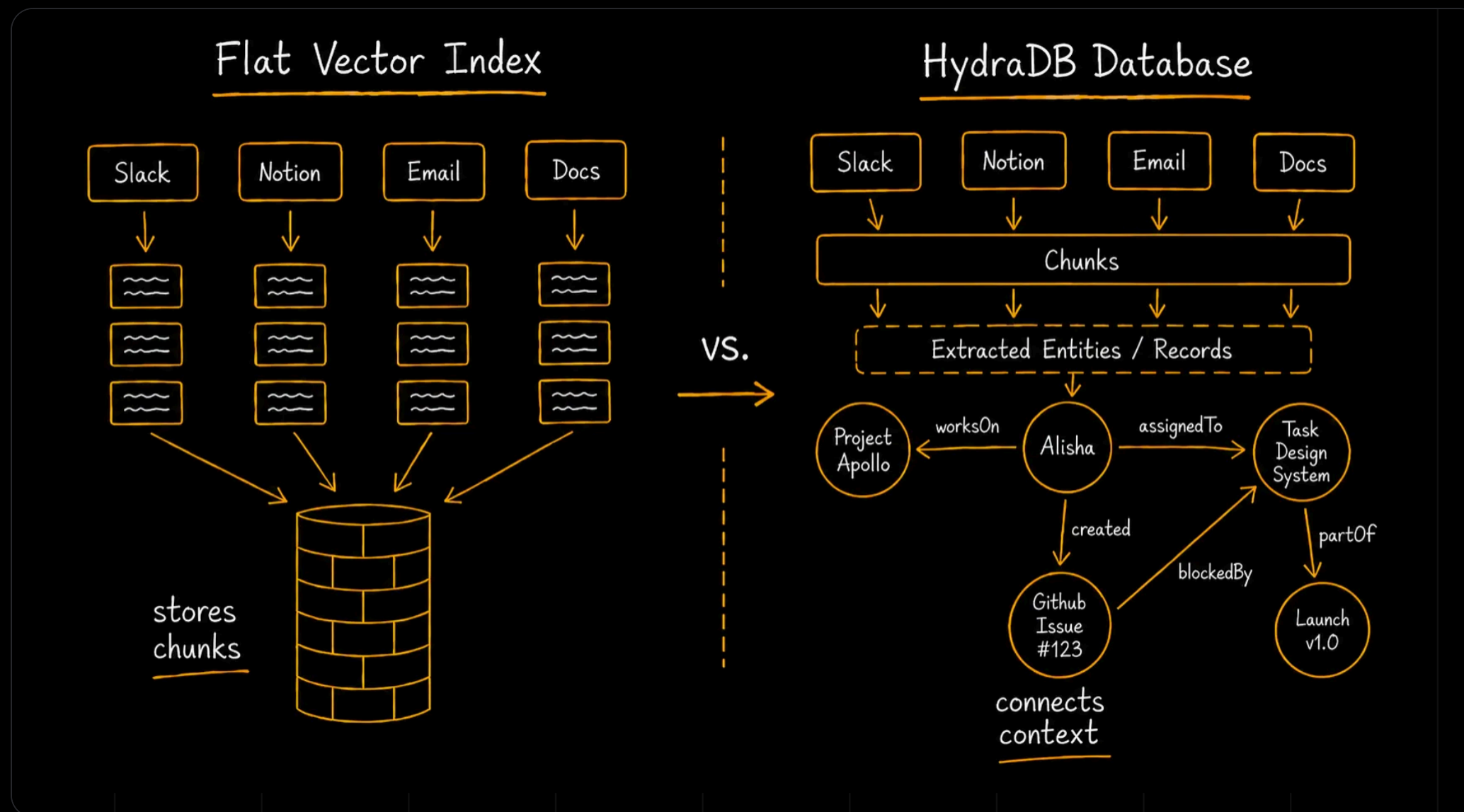
Big-picture questions

“Summarise how my project
evolved” needs hundreds of
chunks, not the top 10. A deep-
research agent can do it, at a
steep cost per question.

WHAT A GRAPH ADDS

Walk one neighbourhood
instead of reading every chunk.

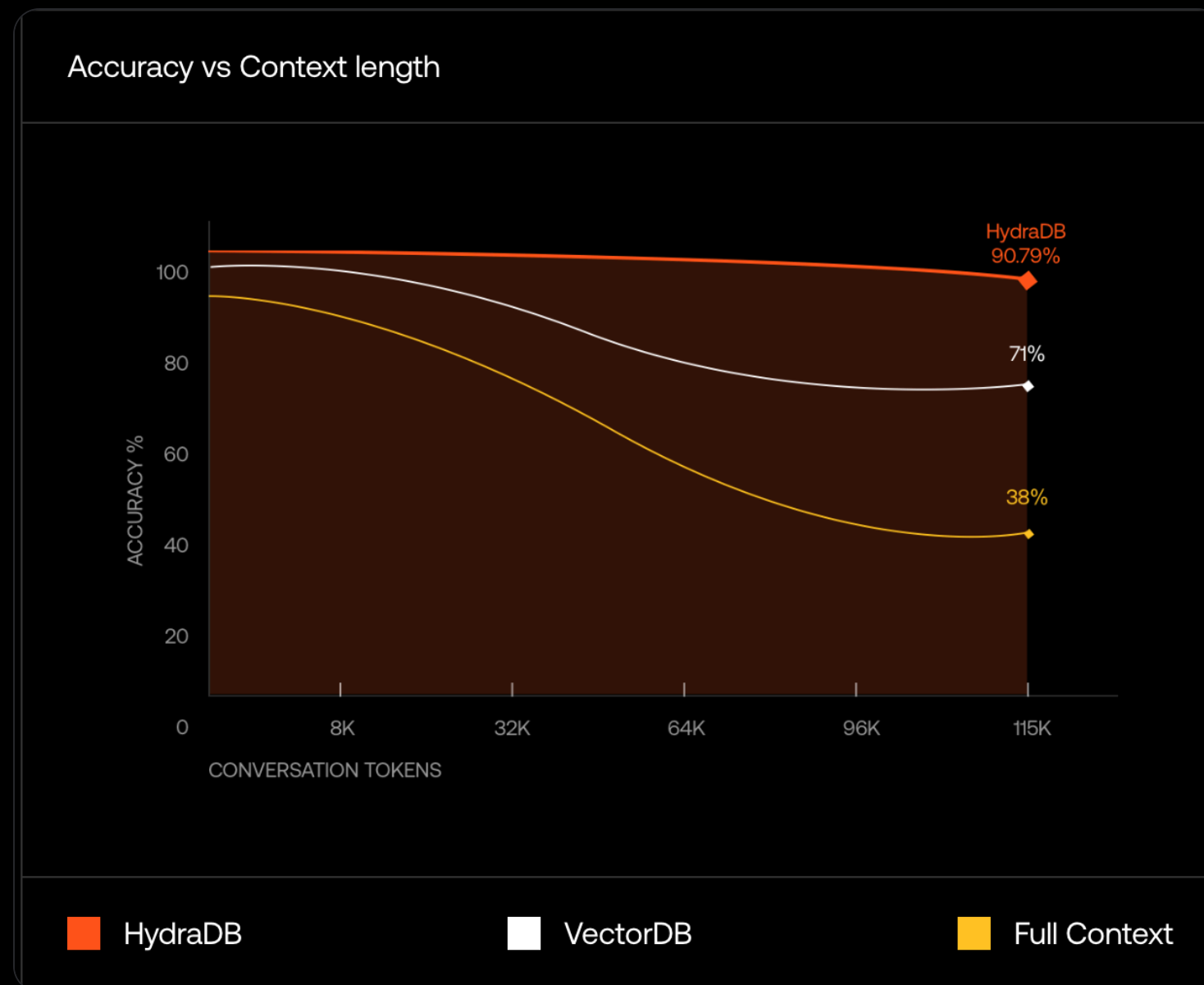
Store chunks vs. connect context



- Every chunk still lands, and entities and relations are extracted from it.
- Facts link across sources and sessions: **who** did **what**, **when**, and what it blocked.
- Retrieval walks the neighbourhood of what the question mentions, instead of ranking isolated chunks.

03 · GRAPH MEMORY AT SCALE

Accuracy that **holds** as memory grows



AT 115K CONVERSATION TOKENS

90.79%

HydraDB: nearly flat from 8K to 115K tokens

71%

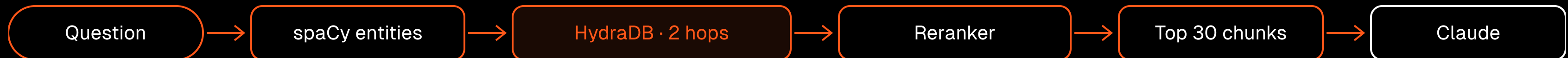
Vector DB: degrades as the store crowds

38%

Full context: the model drowns in its own history

TRY IT YOURSELF

Graph memory on your laptop, in five steps

**1** Clone the repo

```
git clone https://github.com/adityashrivastavaa/hydradb-beam-rag
cd hydradb-beam-rag
```

2 Build HydraDB (pinned, open source)

```
brew install cmake pkg-config llvm suite-sparse \
  cleishm/neo4j/libcypher-parser
HYDRADB_DIR=~/.src/hydradb scripts/install_hydradb.sh
```

3 Start it (own terminal)

```
HYDRADB_DIR=~/.src/hydradb scripts/start_hydradb.sh
```

4 Python env + load the graph

```
scripts/setup_python.sh && source .venv/bin/activate
python -m app.loader # ~2 min, 9,227 relations
```

5 Serve the app

```
uvicorn app.server:app --port 8000
# open http://localhost:8000
```

+ Optional: answers from Claude

```
export ANTHROPIC_API_KEY=... # then "Answer with Claude"
```

ASK THESE NOW

Three questions to watch the graph work

1 Which port did I say my memory store was running on?

`memory store` One clean seed and a small, readable cluster around it. The basic idea in one picture.

2 What retry mechanism with exponential backoff did I use for the OpenAI API calls?

`retry mechanism` `exponential backoff` `openai api` Separate concepts bridged by shared neighbours; the top hits are all about GPT-4 rate-limit retries.

3 How did I design the PostgreSQL database schema for the contextual memory store?

`postgresql database schema` `contextual memory store` A dense neighbourhood; the top hits are PostgreSQL 14 JSONB designs for the store.

Hover a seed to light up its neighbours. The yellow `user` hub is shown but never expanded. Toggle 1 vs 2 hops and watch the pool change.

KEEP EXPLORING

The question bank: **BEAM**

The demo graph is conversation 1 of BEAM-1M, a long-term-memory benchmark (ICLR 2026) whose probing questions test ten memory abilities:

Information extraction

recall a specific fact

Multi-session reasoning

join evidence across sessions

Knowledge update

prefer the newest fact

Temporal reasoning

dates, gaps, deadlines

Event ordering

reconstruct what came first

Contradiction resolution

spot conflicting claims

Preference following

adapt to stated tastes

Instruction following

keep long-lived constraints

Summarization

compress the whole arc

Abstention

say “not in memory”

IN THE APP

The BEAM example questions menu, or
`data/beam_c01/questions.json` (20 questions with
reference answers)

FULL BENCHMARK

huggingface.co/datasets/Mohammadta/BEAM

github.com/mohammadtavakoli78/BEAM

Memory that connects.

Demo repo github.com/adityashrivastavaa/hydradb-beam-rag

HydraDB hydradb.com · open source github.com/hydra-db/hydradb

